



Trabajo Práctico – Unidad 06

Herramientas de cálculo para el taller

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

Objetivos

Al terminar este trabajo práctico vas a poder:

-  Declarar funciones con **arrow function** y llamarlas
 -  Diseñar la **firma**: qué parámetros necesita y qué devuelve
 -  Usar `return` para devolver resultados, incluidos booleanos
 -  Documentar cada función con **JSDoc**
 -  Generar datos con `rndInt(min, max)` y verificar que cubre todo el rango
 -  Convertir un programa largo en uno corto y legible
-

Consignas generales

1. **Todo se resuelve dentro del template de la cátedra.** Escribí en `src/app.js` y ejecutá con `npm run dev`.
2. **Un problema, un archivo.**
3.  **Toda función lleva su JSDoc**, con la descripción, un `@param` por parámetro y el `@returns`. **Se corrige.**
4. **Arrow functions asignadas a `const`**. Nada de `function`.
5. **Cada función hace una sola cosa** y trabaja con **sus parámetros**: no lee variables del programa principal.
5. **Identificadores en inglés**; los textos que ve el usuario, en español.
7. **Cuando el enunciado da una firma, respetala tal cual**: mismo nombre, mismos parámetros, mismo orden. Es lo que se compara al corregir.

 Varios problemas usan números aleatorios, así que **tu salida no va a ser idéntica a la del ejemplo**. Lo que tiene que coincidir es el **formato**.

Problemas

Problema 1 – Paridad

Escribí una función que indique si un número entero es par, y probala con varios valores.

```
const isEven = (n) => { ... }
```

Salida esperada:

```
isEven(4)  → true
isEven(7)  → false
isEven(0)  → true
isEven(-3) → false
```

💡 Devuelve un **booleano**, no un texto. Con el `%` de la unidad 03 entra en una línea.

📌 Las funciones que devuelven booleano se nombran empezando con `is` o `has`.

Problema 2 – Conversión de temperaturas

Escribí **dos** funciones de conversión y una tabla que las use.

```
const toFahrenheit = (celsius) => { ... }
const toKelvin = (celsius) => { ... }
```

$$^{\circ}F = ^{\circ}C \times \frac{9}{5} + 32 \quad K = ^{\circ}C + 273,15$$

Generá con un bucle la tabla de **0 a 100 °C, de 20 en 20**:

Salida esperada:

°C	°F	K
0	32.00	273.15
20	68.00	293.15
40	104.00	313.15
60	140.00	333.15
80	176.00	353.15
100	212.00	373.15

💡 Dos conversiones distintas → **dos funciones**. Si el nombre necesitara un "y", estarías haciendo dos cosas en una.

Problema 3 – Magnitudes eléctricas

Escribí tres funciones y un programa que pida tensión y corriente por consola y muestre las tres magnitudes.

```
const power = (voltage, current) => { ... }  
const resistance = (voltage, current) => { ... }  
const energyPerDay = (watts, hours) => { ... }
```

- Potencia $P = V \times I$, en watt
- Resistencia $R = V/I$, en ohm
- Consumo diario $E = \frac{P \times h}{1000}$, en kWh

Ejemplo de ejecución:

```
Tensión [V]: 380  
Corriente [A]: 4.2  
Horas de uso por día: 8  
  
Potencia:      1596.00 W  
Resistencia:   90.48 Ω  
Consumo diario: 12.77 kWh
```

💡 Fijate que `energyPerDay` recibe **watts**, no tensión y corriente. Cada función pide lo que necesita: para calcular el consumo le pasás el resultado de `power`.

Problema 4 – Clasificar una medición

Escribí una función que reciba una tensión medida y devuelva su clasificación como texto. La norma admite **380 V con ±5 %**.

```
const classify = (measured) => { ... }
```

Salida esperada:

```
classify(300) → BAJA  
classify(365) → NORMAL  
classify(380) → NORMAL  
classify(399) → NORMAL  
classify(405) → ALTA
```

💡 Los `if` con `return` no necesitan `else`: como el `return` corta, si llegaste a la segunda pregunta es porque la primera dio `false`.

⚠️ Los límites van en constantes `UPPER_SNAKE_CASE`, **arriba del archivo**, no adentro de la función.

✅ Probala con `365`, `380` y `399` —los tres tienen que dar `NORMAL`— y con los bordes exactos.

Problema 5 – El dado, y por qué el `+ 1`

Escribí la función `rndInt` de la cátedra y comprobá que **cubre todo el rango**.

```
const rndInt = (min, max) => { ... }
```

El programa tiene que tirar **1000 veces** un dado de 1 a 6 y contar cuántas veces salió cada extremo: el `1` y el `6`.

Ejemplo de ejecución:

```
1000 tiradas de rndInt(1, 6)

salió el 1: 168 veces
salió el 6: 159 veces
```

Después, **a propósito**, escribí una segunda versión sin el `+ 1` y comprobá qué pasa:

```
1000 tiradas SIN el + 1

salió el 1: 201 veces
salió el 6: 0 veces ← nunca
```

🔑 **Ése es el punto del problema.** El programa sin el `+ 1` anda, no da error, y los números parecen razonables. Simplemente **falta uno** de todos los valores posibles. Es el tipo de error que sólo se encuentra si lo buscás.

Problema 6 – Simulación de una jornada

Usando `rndInt`, simulá una jornada de mediciones de tensión y contá cuántas quedaron fuera de tolerancia.

```
const rndInt = (min, max) => { ... }
const simulateMeasurement = () => { ... }
const isWithinRange = (measured) => { ... }
```

- `simulateMeasurement` genera una tensión entre **355 y 405 V**, sin parámetros
- `isWithinRange` devuelve un booleano: si está dentro de 380 V $\pm 5\%$

Pedí por consola cuántas mediciones simular y mostrá cada una, marcando las que se van.

Ejemplo de ejecución:

```
¿Cuántas mediciones simular? 6

Medición 1: 371 V
Medición 2: 357 V ←
Medición 3: 380 V
Medición 4: 402 V ←
Medición 5: 388 V
Medición 6: 364 V

Fuera de rango: 2 de 6 (33.3 %)
```



`simulateMeasurement` no lleva parámetros: los paréntesis van igual, vacíos.

Problema 7 – Informe de jornada, con funciones

Integra todo lo anterior.

Rehacé el **problema 7 del TP de la unidad 05** —el informe de la jornada— pero ahora con funciones, y con las mediciones **simuladas** en vez de tipeadas.

El informe muestra lo mismo que antes: cantidad, promedio, máxima, mínima, cuántas fuera de tolerancia con su porcentaje, y el veredicto.

Las funciones que tenés que escribir, como mínimo:

```
const rndInt = (min, max) => { ... }
const simulateMeasurement = () => { ... }
const isWithinRange = (measured) => { ... }
const deviation = (measured) => { ... }
const printLine = (index, measured) => { ... }
```

Ejemplo de ejecución:

```
¿Cuántas mediciones simular? 5
```

```
Medición 1: 371 V    -2.37 %  
Medición 2: 357 V    -6.05 % ←  
Medición 3: 380 V     0.00 %  
Medición 4: 402 V     5.79 % ←  
Medición 5: 388 V     2.11 %
```

```
— Informe de la jornada —
```

```
Mediciones:      5  
Promedio:        379.60 V  
Máxima:          402.00 V  
Mínima:          357.00 V  
Fuera de rango:  2 (40.0 %)
```

```
Veredicto:      NO CONFORME
```

💡 **Compará con lo que escribiste en la unidad 05.** El programa principal tendría que quedarte en un bucle y unas pocas líneas de informe: todo lo demás vive adentro de las funciones.

💡 Los patrones de la unidad 05 —acumulador, contador, máximo, mínimo— siguen estando. Lo que cambia es que ahora las **decisiones** de cada uno están encapsuladas en una función con nombre.

⚠️ El máximo y el mínimo se inicializan con la **primera medición**, no en cero. Como la primera se genera dentro del bucle, un `if (i === 1)` resuelve el arranque.

🚫 Restricciones

- ❌ **Sin vectores ni arrays.** Se ven en la **unidad 07**.
- ❌ **Sin `function`:** arrow functions asignadas a `const`.
- ❌ **Sin devolver objetos ni JSON.** Sólo primitivos: número, texto, booleano.
- ❌ **Sin funciones que lean variables del programa principal** — salvo las constantes en `UPPER_SNAKE_CASE`, que sí se pueden usar.
- ❌ **Sin `var`,** sin `==`, sin punto y coma, sin condicionales de una línea sin llaves.
- ❌ **Sin función sin JSDoc.**
- ✅ Lo de esta unidad y las anteriores.

😬 **Lo que vas a sentir en el problema 7:** que ya está mucho mejor que en la unidad 05, pero que sigue habiendo algo raro — **las mediciones se generan y se pierden**. No podés volver a mirarlas, ni ordenarlas, ni mostrar las tres más altas.

Eso es lo último que falta, y llega la clase que viene.



Material de consulta

- [Apunte de la unidad](https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/apunte.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/apunte.pdf)
- [Presentación de clase](https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/presentacion.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/06-funciones/presentacion.pdf)
- [Ejemplos de la clase](https://github.com/italijancic/pc-2026/tree/main/unidades/06-funciones/ejemplos) (https://github.com/italijancic/pc-2026/tree/main/unidades/06-funciones/ejemplos)
- [Template del curso](https://github.com/italijancic/pc-2026/tree/main/template) (https://github.com/italijancic/pc-2026/tree/main/template)